Lộ trình học: Point Tracking, Pose Refinement, Mask Optimization

Oct 28, 2026 · @Hadinho

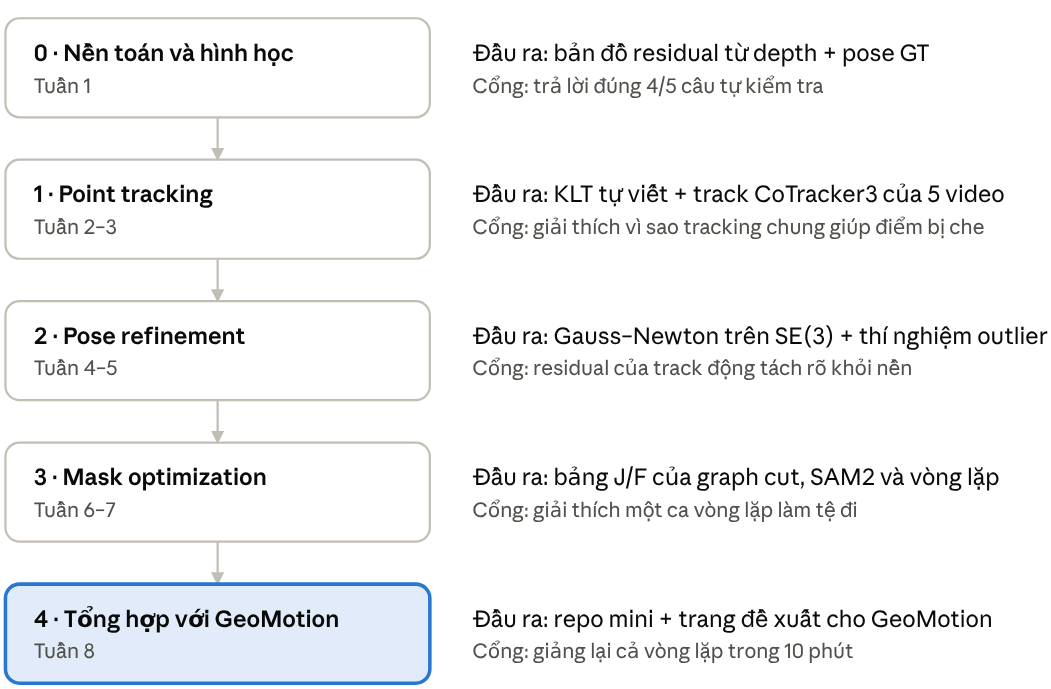
Tổng quan

Sau 8 tuần, bạn sẽ tự cài đặt được một vòng lặp nhỏ: track điểm → ước lượng pose camera → suy ra mask chuyển động → tinh chỉnh mask, và chạy nó trên vài video DAVIS. Đây là cách học "bản chất": hiểu toán, tự code phiên bản đơn giản, rồi mới đọc paper hiện đại.

Giả định (sửa nếu sai): 8 tuần, 10–12 giờ/tuần; bạn đã vững PyTorch, đại số tuyến tính cơ bản và đã đọc RAFT qua GeoMotion; có 1 GPU để chạy inference CoTracker3 và SAM2.

Ý xuyên suốt: cả ba chủ đề đều là *iterative refinement*: có ước lượng thô → so với quan sát → tính sai số → cập nhật. Khác nhau chỉ ở biến được cập nhật (vị trí điểm, pose SE(3), hay mask) và hàm sai số.

Chủ đề	Biến cập nhật	Sai số đo bằng	Ví dụ kinh điển → hiện đại
Point tracking	Tọa độ (x, y) của điểm qua T frame	Tương quan feature quanh điểm	Lucas–Kanade → PIPs → CoTracker3
Pose refinement	Pose camera $T \in SE(3)$ (+ depth)	Sai số tái chiếu	Gauss–Newton, BA → DROID-SLAM → MegaSaM
Mask optimization	Mask nhị phân / xác suất	Năng lượng: residual chuyển động + trơn + bám biên	Graph Cut, CRF → lặp mask↔pose → prompt SAM2



lộ trình 8 tuần · 5 giai đoạn, mỗi giai đoạn có cổng

Mỗi giai đoạn dùng lại đầu ra của giai đoạn trước, nên đừng nhảy cóc: track của tuần 3 là đầu vào cho pose, residual của tuần 5 là đầu vào cho mask.

Mục tiêu cuối khóa

Cuối tuần 8, bạn đạt khi làm được cả ba nhóm dưới đây mà không cần mở lại tài liệu.

Hiểu (giải thích được bằng lời và bằng công thức)

- Viết ra bài toán Gauss–Newton cho một hàm sai số bất kỳ và giải thích vì sao Levenberg–Marquardt ổn định hơn.
- Giải thích vì sao pose được cập nhật bằng $T \leftarrow \exp(\delta\xi)\cdot T$ thay vì cộng trực tiếp vào ma trận.
- Phân biệt optical flow, point tracking và scene flow; nêu 3 khó khăn chính của tracking dài (che khuất, trôi, ra khỏi khung).
- Giải thích bài toán "con gà – quả trứng": cần mask để ước lượng pose sạch, cần pose để suy ra mask.

Làm (có code chạy được)

- KLT tracker tự viết bằng NumPy, chạy trên 1 video DAVIS.
- Bộ giải Gauss–Newton tìm pose camera từ tương ứng 2D–3D, có trọng số robust (Huber).
- Một pipeline: CoTracker3 → ước lượng chuyển động camera → phân loại track động/tĩnh → prompt SAM2 → mask.

Sản phẩm đo được

- 3 bài ghi chú (mỗi chủ đề 1 bài, 2–3 trang), viết theo mẫu ở cuối tài liệu.
- Repo mini có kết quả J/F trên ít nhất 5 video DAVIS16, so với baseline "chỉ dùng độ lớn flow" (như GeoMotion bản CVPR).
- Một trang đề xuất: có nên đưa residual flow hoặc vòng lặp pose↔mask vào GeoMotion không, và bằng chứng từ mini-project.

Giai đoạn 0 — Tuần 1: Nền toán và hình học

Mục tiêu của tuần: nắm đủ công cụ để đọc mọi phương trình ở các giai đoạn sau. Đây là tuần quan trọng nhất; đừng bỏ qua dù thấy quen.

Nội dung

1. Mô hình camera pinhole: ma trận nội tại K, ngoại tại [R|t], chiếu điểm 3D xuống ảnh và back-project bằng depth.
2. Phép quay và SE(3): ma trận quay, Lie algebra so(3)/se(3), ánh xạ exp/log, nhiễu động nhỏ $\delta\xi$.
3. Bình phương tối thiểu phi tuyến: Gauss–Newton, Levenberg–Marquardt, ma trận Jacobian, hàm mất mát robust (Huber, Cauchy) và IRLS.
4. Optical flow và hình học epipolar: flow gây ra bởi camera = f(depth, pose); phần còn lại là chuyển động của vật.

```
u_{cam} = \pi\big(K \, T_{j \leftarrow i} \, D_i(u) \, K^{-1} \tilde{u}\big)
- u, \qquad r(u) = u_{obs} - u_{cam}
```

Phương trình trên là xương sống của cả khóa: flow do camera tính từ depth và pose; residual r lớn → điểm đó có khả năng đang chuyển động.

Đọc

- Szeliski, *Computer Vision: Algorithms and Applications* (2nd ed., bản PDF miễn phí): chương 2 (hình thành ảnh), mục về tối ưu phi tuyến trong phụ lục A.

- Hartley & Zisserman, *Multiple View Geometry*: chương 6 (camera), chương 9 (epipolar), phụ lục 6 (LM).
- Solà, Deray, Atchuthan, *A micro Lie theory for state estimation in robotics* (2018): đọc mục 1–4.

Bài tập

- [ ] Tự viết hàm `project(K, T, X)` và `backproject(K, depth, uv)` bằng NumPy, kiểm tra ngược lại cho khớp.
- [ ] Viết `exp_se3` / `log_se3`, kiểm tra log(exp(ξ)) = ξ với ξ ngẫu nhiên.
- [ ] Dùng Gauss–Newton fit một đường cong y = a·exp(b·x) có nhiễu và outlier; so sánh L2 với Huber.
- [ ] Từ 1 frame Sintel hoặc Dynamic Replica có depth + pose GT: tính u_cam, so với flow GT, vẽ bản đồ residual. Vật động phải nổi lên rõ.

Tự kiểm tra (trả lời được 4/5 câu mới sang tuần 2)

1. Vì sao không thể cập nhật ma trận quay bằng $R \leftarrow R + \Delta R$?
2. Ma trận J^TJ trong Gauss–Newton xấp xỉ cái gì, và khi nào xấp xỉ đó tệ?
3. Camera chỉ xoay, không tịnh tiến: flow do camera có phụ thuộc depth không? Điều này ảnh hưởng gì đến việc phát hiện vật động?
4. Huber loss làm gì với outlier, nói theo ngôn ngữ trọng số IRLS?
5. Vì sao GeoMotion chỉ dùng độ lớn flow lại khó phân biệt vật động với tiền cảnh gần camera?

Giai đoạn 1 — Tuần 2–3: Point tracking

Mục tiêu: hiểu vì sao tracking hiện đại = "tìm kiếm toàn cục + tinh chỉnh lặp cục bộ", và dùng thành thạo CoTracker3 làm công cụ.

Tuần 2 — Từ cổ điển đến PIPs

- Đọc: Szeliski chương 9 (Lucas–Kanade, pyramid, KLT). Ôn lại RAFT (correlation volume + cập nhật GRU lặp).
- Đọc: PIPs — Particle Video Revisited (ECCV 2022). Tập trung mục 3: khởi tạo quỹ đạo, correlation đa frame, cập nhật lặp, visibility.
- [ ] Viết KLT bằng NumPy (Lucas–Kanade 1 cấp rồi thêm pyramid), track 50 điểm trên 1 video DAVIS.

- [ ] Ghi lại 3 trường hợp KLT thất bại (che khuất, chuyển động nhanh, vùng thiếu texture) bằng ảnh chụp.
- [ ] Vẽ sơ đồ tay kiến trúc PIPs, chỉ ra chỗ nó giống RAFT và chỗ khác.

Tuần 3 — Bài toán TAP và tracker hiện đại

- Đọc: TAP-Vid (NeurIPS 2022) — định nghĩa bài toán và metric AJ, δ_avg, OA.
- Đọc: TAPIR (ICCV 2023) — giai đoạn matching + refinement.
- Đọc: CoTracker3 (ICCV 2025) — tracking chung nhiều điểm, pseudo-label trên video thật.
- [ ] Cài CoTracker3, chạy grid 30×30 trên cùng video DAVIS, so với KLT của bạn.
- [ ] Thử: tắt cross-track attention (chạy từng điểm riêng, cờ single_point) — điểm bị che có tệ hơn không?
- [ ] Lưu track + visibility của 5 video DAVIS16 ra file .npz — đây là đầu vào cho giai đoạn 2 và 3.

Tự kiểm tra

1. Optical flow và point tracking khác nhau ở điểm nào? Vì sao nối flow 2 frame liên tiếp lại bị trôi?
2. Vì sao tracking chung nhiều điểm giúp điểm bị che?
3. AJ trong TAP-Vid đo đồng thời hai thứ gì?
4. Một track dài chứa thông tin gì về chuyển động mà độ lớn flow 1 cặp frame (như GeoMotion) không có?

Đầu ra: bài ghi chú số 1 + thư mục `tracks/` cho 5 video.

Giai đoạn 2 — Tuần 4–5: Iterative pose refinement

Mục tiêu: tự cài đặt được vòng lặp tối ưu pose, và hiểu vì sao vật chuyển động làm hỏng nó. Khóa này tập trung vào **pose camera**; pose 6D vật thể chỉ đọc để so sánh.

Tuần 4 — Tối ưu pose cổ điển

- Đọc: Triggs et al., *Bundle Adjustment — A Modern Synthesis* (2000), mục 1–4 và 6 (robust cost).
- Đọc nhanh: DeepIM (ECCV 2018) — cùng ý "so sánh → dự đoán Δpose → lặp", nhưng cho vật thể.
- [ ] Viết Gauss–Newton trên SE(3) cho bài toán PnP: biết điểm 3D (từ depth GT) và vị trí 2D ở frame khác, tìm pose.

- [ ] Thí nghiệm: trộn 0%, 20%, 40% điểm trên vật động vào đầu vào. Đo sai số pose với L2, Huber, và RANSAC + Huber. Ghi kết quả vào bảng.

Tuần 5 — Pose học sâu trong video động

- Đọc: DROID-SLAM (NeurIPS 2021) — lớp Dense BA, cách confidence weight chạy vào BA.
- Đọc: MegaSaM (CVPR 2025) — bản đồ xác suất chuyển động giảm trọng số trong BA; đọc kỹ phần giới hạn.
- Đọc nhanh: CasualSAM (ECCV 2022) — test-time optimization.

- [ ] Dùng track từ giai đoạn 1 + depth/pose của π^3 (đã có trong GeoMotion): tính residual cho từng track, vẽ histogram residual của track tĩnh và động (dùng mask GT DAVIS để gán nhãn).
- [ ] Thử vòng lặp: ước lượng pose → bỏ track residual cao → ước lượng lại. Sau 3 vòng, residual của vật động tách khỏi nền rõ hơn không?

Tự kiểm tra

1. Trong DROID-SLAM, cái gì được học và cái gì vẫn là tối ưu tường minh?
2. Vì sao khi vật động chiếm phần lớn khung hình, mọi phương pháp robust đều có thể hỏng?
3. Chuyển động vật thẳng hàng với chuyển động camera gây ra mơ hồ gì? Liên hệ với các ca khó trong FBMS.
4. π^3 trong GeoMotion dự đoán pose bằng 1 lần forward. Nếu thêm vòng tinh chỉnh, bạn sẽ đặt nó ở đâu?

Đầu ra: bài ghi chú số 2 + bảng kết quả thí nghiệm outlier + hàm `residual_per_track()` dùng lại ở giai đoạn 3.

Giai đoạn 3 — Tuần 6–7: Mask optimization

Mục tiêu: biến tín hiệu chuyển động thưa (residual trên từng track) thành mask dày đặc, bám biên, nhất quán theo thời gian; hiểu ba trường phái và đánh đổi giữa chúng.

Tuần 6 — Mask như bài toán năng lượng

- Đọc: Boykov & Jolly (2001) về graph cut tương tác; Rother et al., *GrabCut* (2004); Krähenbühl & Koltun, *DenseCRF* (2011). Chỉ cần nắm: unary term, pairwise term, vì sao min-cut giải đúng.

```
E(m) = \sum_u \big[m_u \, \phi_{dyn}(u) + (1 - m_u) \, \phi_{stat}(u)\big] +
\lambda \sum_{(u,v)} w_{uv} \, [m_u \neq m_v]
```

- [ ] Từ residual của track (giai đoạn 2), nội suy ra bản đồ động thô cho cả ảnh, rồi chạy graph cut (thư viện PyMaxflow) với w_uv theo màu. Đo J trên 5 video.
- [ ] Thí nghiệm λ: vẽ J theo λ, giải thích hai đầu cực (mask lỗm đỗm vs mask nuốt vật).

Tuần 7 — Vòng lặp mask ↔ pose, và prompt SAM2

- Đọc: Segment Any Motion in Videos (CVPR 2025) + supplementary (pseudo-code prompt lặp).
- Đọc nhanh: ROS-Cam (NeurIPS 2025) — tối ưu đồng thời, giảm trọng số outlier, bẫy cực tiểu địa phương.

- [ ] Dùng track đã phân loại động làm point prompt cho SAM2; so với mask graph cut tuần 6.
- [ ] Vòng lặp đầy đủ: mask → loại điểm trong mask khi tính pose → residual mới → mask mới. Chạy 1, 2, 3 vòng, ghi J mỗi vòng.
- [ ] Tìm 1 video mà vòng lặp làm tệ đi, giải thích vì sao.

Tự kiểm tra

1. Graph cut và SAM2 đều cho mask bám biên. Mỗi cái lấy thông tin biên từ đâu?
2. Vì sao vòng lặp mask ↔ pose có thể hội tụ về lời giải sai? Điều kiện khởi tạo nào giúp tránh?
3. SegAnyMotion prompt lại SAM2 theo chu kỳ. Nó khắc phục điểm yếu nào của bộ nhớ SAM2?
4. GeoMotion chọn chế độ single/multi theo tên dataset. Một cơ chế dựa trên track sẽ thay thế việc đó thế nào?

Đầu ra: bài ghi chú số 3 + bảng J/F của 3 cách (graph cut, SAM2 prompt, vòng lặp) trên 5 video.

Giai đoạn 4 — Tuần 8: Tổng hợp và nối với GeoMotion

Mục tiêu: trả lời bằng số liệu câu hỏi "residual chuyển động và vòng lặp pose↔mask có đáng đưa vào GeoMotion không?".

- [ ] Gom code 3 giai đoạn thành 1 repo mini: `track.py`, `pose.py`, `mask.py`, `run_davis.py`.

- [ ] Chạy baseline "độ lớn flow RAFT + ngưỡng" (mô phỏng tín hiệu GeoMotion bản CVPR dùng) trên cùng 5 video.
- [ ] Thêm 2 video FBMS có chuyển động yếu — đây là nơi GeoMotion thua (J = 72.5).
- [ ] Bảng so sánh cuối: baseline flow magnitude, residual + graph cut, residual + SAM2, vòng lặp — cột J, F, thời gian/frame.
- [ ] Viết 1 trang đề xuất: nên thêm gì vào GeoMotion (ví dụ residual flow làm kênh F_flow mới, hoặc vòng tinh chỉnh sau mask head), rủi ro, và thí nghiệm tiếp theo.

Tiêu chí hoàn thành khóa

- Giảng lại được cả vòng lặp trong 10 phút cho bạn cùng nhóm, không nhìn slide.
- Bảng so sánh có kết luận rõ, kể cả khi kết luận là "không cải thiện".
- Đọc một paper mới về motion segmentation và chỉ ra nó thuộc trường phái nào trong 3 trường phái mask, dùng tracker nào, tính pose ra sao.

Cách học và theo dõi tiến độ

Nhịp mỗi tuần (khoảng 11 giờ)

- 4 giờ đọc, 5 giờ code/thí nghiệm, 1 giờ viết ghi chú, 1 giờ tự kiểm tra.
- Cuối tuần gửi tôi: câu trả lời phần Tự kiểm tra + 1 hình kết quả. Tôi sẽ chấm và chỉ ra chỗ hiểu sai.

Cách đọc một paper (3 lượt)

1. Lượt 1 (15 phút): abstract, hình 1–2, kết luận. Viết 1 câu: paper giải quyết vấn đề gì, bằng ý chính nào.
2. Lượt 2 (1–2 giờ): phương pháp. Chép tay mọi công thức chính, ghi biến nào được cập nhật và hàm sai số là gì.
3. Lượt 3 (1 giờ): ablation và giới hạn. Tự hỏi: nếu bỏ thành phần X thì tệ bao nhiêu, và ca nào phương pháp chắc chắn hỏng.

Mẫu ghi chú cho mỗi chủ đề

```
# <Chủ đề>
## Bài toán: đầu vào, đầu ra, khó ở đâu (3 dòng)
## Biến cập nhật và hàm sai số (công thức)
## Lịch sử: cổ điển -> học sâu -> hiện tại (mỗi bước 1 câu: cải tiến gì)
## Kết quả thí nghiệm của tôi (bảng + 1 hình)
## Liên hệ GeoMotion: áp vào đâu, rủi ro gì
## Câu hỏi còn mở
```

Bảng tiến độ

Giai đoạn	Tuần	Sản phẩm phải nộp	Trạng thái
0. Nền toán	1	4 bài tập + bản đồ residual	Chưa bắt đầu
1. Point tracking	2–3	Ghi chú 1 + `tracks/` 5 video	Chưa bắt đầu
2. Pose refinement	4–5	Ghi chú 2 + bảng thí nghiệm outlier	Chưa bắt đầu
3. Mask optimization	6–7	Ghi chú 3 + bảng J/F 3 cách	Chưa bắt đầu
4. Tổng hợp	8	Repo mini + trang đề xuất	Chưa bắt đầu

Nguồn

- CoTracker3 — arXiv · GitHub
- TAP-Vid — arXiv
- TAPIR — arXiv
- PIPs — arXiv
- DROID-SLAM — arXiv
- MegaSaM — arXiv
- CasualSAM — ECCV 2022
- DeepIM — arXiv
- ROS-Cam — NeurIPS 2025
- Segment Any Motion in Videos — arXiv
- Sách và bài cổ điển (Szeliski, Hartley & Zisserman, Triggs, Solà, Boykov, GrabCut, DenseCRF) được nêu từ trí nhớ, chưa dẫn link; số chương có thể lệch tùy ấn bản.